

Machine learning in production

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build Project 1, `nw/triage`: a calibrated four-class priority classifier behind a FastAPI service, and the loop that keeps it honest. A data contract, a promotion gate, a run log and a model card, drift monitoring, a shadow slot for the next version, and a retraining workflow that never promotes on its own.

Where we are

PART 1

Foundations and
the AI service layer

PART 2

Machine learning in
production

PART 3

Deep learning and
representation

PART 4

Generative AI:
retrieval and
evaluation

PART 5

Agentic AI systems

PART 6

Deployment,
observability and
capstone

Project 1 orders Northwind's queue by priority. The agent calls it as a tool; the capstone puts it in front of the agent as a cheap pre-filter and turns its drift line into an alarm. It has to be right about P0 far more often than about anything else.

Today, step by step

- 1 Read the data and the contract
- 2 The naive model meets the gate
- 3 Class weights
- 4 A threshold, not argmax
- 5 Calibrate the probabilities
- 6 Track and compare runs
- 7 Read the model card and the promotion log
- 8 Alive is not ready, and the service knows when the world changes
- 9 Shadow the next version **OPTIONAL**
- 10 Build the container
- 11 The retraining workflow **OPTIONAL**

Read the P0 row first

Step 2 trains a model that passes 18 of 29 tests and fails the gate. Do not fix it until you have read what it gets wrong and what the gate said.

At the top of step 8, start `make image-triage` in a second terminal. The build takes minutes and step 10 needs it.

Accuracy is a trap on four percent P0

3.9%

of Northwind's tickets are P0. A model that never predicts P0 is 96 percent accurate on that class and useless

Metric	What it answers	Why it matters here
Accuracy	How often is the top guess right?	Dominated by the common classes; hides P0
Macro-F1	Average F1 across four classes, equally weighted	Rewards getting the rare class right
P0 recall	Of true P0s, how many were flagged?	The commercial number: a missed P0 is an SLA breach
P0 precision	Of flagged P0s, how many were real?	The cost of the recall you buy
Brier and ECE	Do the probabilities mean what they say?	A threshold on a bad probability is a threshold on noise

Nothing trains until the data passes its contract

Expectation	What it catches
Every row matches the pydantic contract	a renamed column, an empty body
P0 between 2 and 8 percent	an export that drops or floods the rare class
Validation and test at least 8 percent each	numbers that rest on too few rows
Duplicates under 0.5 percent, no split leak	a test number that is a lie
Every label in every split	a sweep or a calibration that cannot run

`make data-check`

rows 7950, splits 6361 / 795 / 794, P0 3.9%, de 4.9%, text length p10 to p90 401 to 642, duplicates 0, **DATA OK.**

The same run writes the data profile. It ships inside every artifact and is what the service measures drift against.

The naive model sends 13 real emergencies to the ordinary queue

18 of 31

true P0 tickets caught by argmax on the test split. The P0 row reads 18, 13, 0, 0

- Accuracy 0.694, macro-F1 0.688, against a majority baseline of 0.414 and 0.146: clearly learning something
- 31 real P0s in the test split; 13 predicted P1, where they wait behind ordinary work
- P0 precision 0.900: when it says P0 it is usually right, which is the wrong trade for an outage
- Then the gate: `GATE FAILED`, `P0 recall 0.581` is below the bar `0.85`, and 0.03 below production
- `make session02`: 4 failed, 18 passed, 7 errors

A candidate becomes the served model only by passing the gate

Bar	Value	What it protects
P0 recall, absolute	at least 0.85	Northwind's SLA, whatever the last model did
ECE, absolute	at most 0.12	the threshold means something
Macro-F1 against production	drop of at most 0.02	recall not bought by wrecking three classes
P0 recall against production	drop of at most 0.03	no quiet regression on the commercial number
Brier against production	increase of at most 0.01	probabilities not getting worse
Same test split	identical data hash	two numbers that mean the same thing

Production is `artifacts/triage/latest`, or the committed summary in `data/golden/` when there is none. Pass: `latest` moves, the registry alias moves, the summary is rewritten, one line goes to `promotions.jsonl`. `--force` is allowed and recorded.

Three fixes for imbalance, in this order

1. Class weights

Charge a P0 mistake by the inverse of its frequency, about ten times a P2 mistake.

Moves the boundary.

Probabilities stay biased.

2. A threshold

Predict P0 whenever its probability clears a value chosen on validation to hit a recall target.

The business decision lives here.

3. Calibration

Map scores to real probabilities on held-out data, so 0.3 means three in ten are real.

Makes the threshold honest.

Resampling is a fourth option. On short text with a linear model it rarely beats class weights and it makes the validation split lie, so it is not in the acceptance criteria.

Each fix meets the gate, and only the third gets through

Step	Threshold	Macro - F1	P0 recall	P0 precision	The gate says
2 as shipped	0.50	0.688	0.581	0.900	recall below the bar
3 class weights	0.50	0.717	0.774	0.828	recall below the bar
4 threshold	0.05	0.635	0.935	0.377	macro-F1 drops more than 0.02 from 0.663
5 calibration	0.05	0.663	0.903	0.467	PROMOTED

Never on the test split. The sweep runs on validation. The test split is the number you report; choosing on it makes that number a lie.

Calibration made two numbers worse and the decision better

28 of 31

P0 caught at threshold 0.05 after calibration. False P0 alarms from the P1 row: 38 down to 28

- Brier (P0) 0.0115 to 0.0140, ECE 0.058 to 0.091: slightly worse, and the report says so
- Argmax P0 recall falls to 0.677: the sigmoid map pulls the raw P0 scores towards the truth, and P0 is rare
- At 0.05, ten P1 tickets drop below the line: macro-F1 0.663, recall 0.903, and the gate passes

Calibration and the threshold rule go together. With plain argmax, calibration would have cost recall. With the rule, it bought precision, and that is what got the candidate past production.

A run is not a promotion

weights	calibrated	threshold	macro - F1	P0 recall	P0 precision	served
none	no	0.50	0.688	0.581	0.900	
balanced	no	0.50	0.717	0.774	0.828	
balanced	no	0.05	0.635	0.935	0.377	
balanced	yes	0.05	0.663	0.903	0.467	latest
balanced	yes, floor 0.5	0.09	0.687	0.903	0.560	

`make runs` reads `artifacts/triage/runs.jsonl`; the same runs sit in MLflow as the experiment `triage` and the registered model `northwind-triage`, aliases `candidate` and `production`. The fifth row would pass the gate. It is not served because nobody decided it should be.

The card says what the report did not

1 of 2

German P0 tickets caught in the test split. 39 German rows, macro-F1 0.555 against 0.668 for English

`MODEL_CARD.md` is generated from the artifact's own metadata, so it cannot go stale:

- intended use, and what it is not for
- the model, the threshold and the two targets, the data hash, the row counts
- the test table, then the same numbers per language
- the training data shares, limitations, and how it is monitored

`promotions.jsonl` beside it: every gate decision, with reasons, and `"forced": true` when someone overrode one. That is the line a reviewer looks for first.

Six endpoints, and a model that fails to load leaves the process alive and not ready

Endpoint	Answers	Who calls it
<code>/healthz</code>	Is the process up?	the liveness probe; failing it restarts
<code>/readyz</code>	Can it score right now?	the load balancer; failing it stops traffic
<code>/trriage</code>	prediction, probabilities, rule, version	the agent, the capstone router
<code>/metrics</code>	counts, latency, version, drift, shadow	the scrape target
<code>/version</code>	the artifact's metadata, the shadow version	"which model is this"
<code>/drift</code>	PSI against training, and a level	"is this still the world it was trained on"

Load fails

Process stays up. `state.model = None`, `ready = False`, `MODEL_LOADED 0`, one log line with the path and the error.

`/healthz` 200. `/readyz` and `/trriage` 503 until a model is loaded. Crashing loops the restart and scrolls the one useful line off the screen.

The service knows when the world changes

8.87

text-length PSI after fifty one-line tickets. Below 0.1 stable, 0.1 to 0.2 watch, above 0.2 alert

- Two signals in the request path: text length against the profile's histogram, predicted priority against the training shares
- A sliding window of 500, nothing said before 50, recomputed every 50
- `nw_triage_drift_psi{feature}`, `nw_triage_drift_level`, and one `drift_alert` log line past 0.2

AWS

A metric filter on the triage function's log group counts `drift_alert`; an alarm on the first one pages SNS.

Google Cloud

A log-based metric on the same line; an alert policy on the first one emails the channel.

The next version runs beside the served one first

`NW_TRIAGE_SHADOW_MODEL` scores every request with a second model and serves none of it.

`NW_TRIAGE_CAPTURE` keeps the requests for a backtest.

Twenty real P0 tickets through a candidate trained for recall 0.95, which fell back to threshold 0.48:

15 / 5

`nw_triage_shadow_total{agree="true"}` and `{agree="false"}`. Every disagreement: served P0, shadow P1

<code>make backtest-triage</code> on the test split	Macro-F1	P0 recall	Predicted P0 share
served, threshold 0.05	0.663	0.903	0.08
candidate, threshold 0.48	0.716	0.677	0.03

Agreement 0.953; disagreements `P0->P1 33, P0->P2 4`. `make promote-triage`: `GATE FAILED`, recall below the bar.

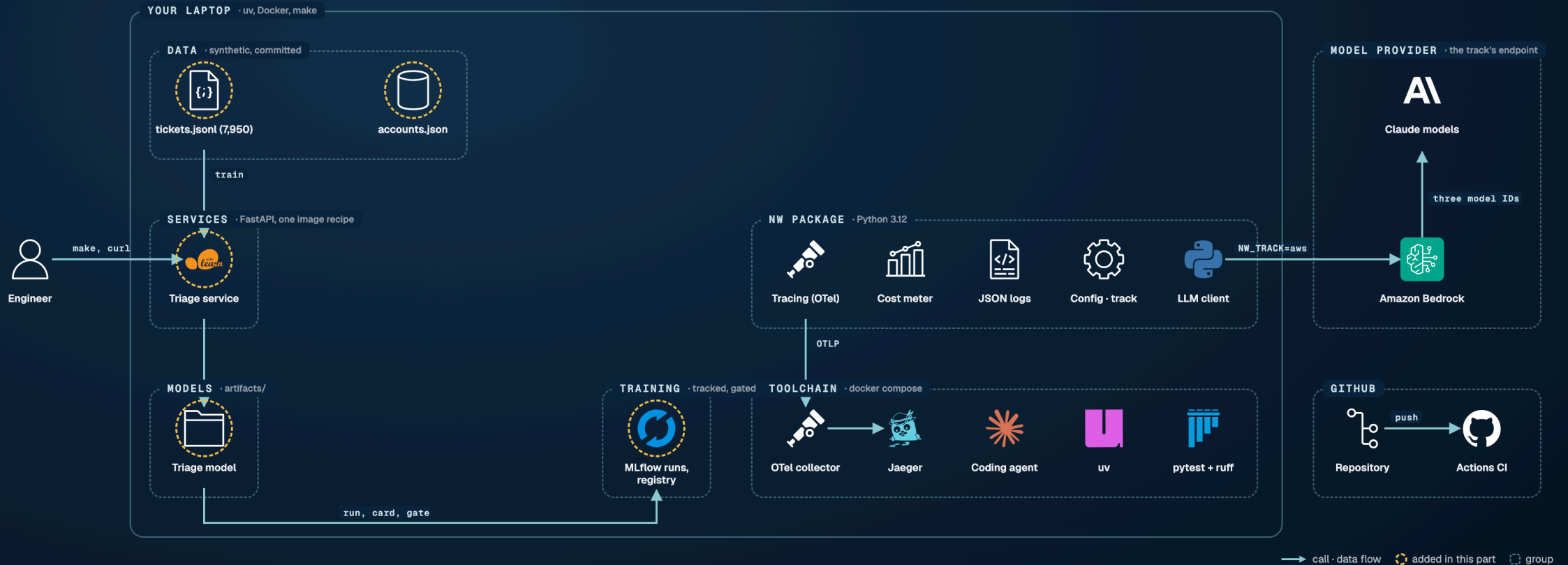
The workflow retrains every Monday and never promotes

- 1 Mondays 05:00 UTC, on demand, and on any pull request that touches `data/tickets.jsonl` or `nw/triage`
- 2 `data_check`, then `train --no-promote`
- 3 The gate against the committed summary `data/golden/triage_production.json`, because CI has no `latest`
- 4 The report, the gate line and the model card in the job summary; the candidate as a workflow artifact for 30 days
- 5 The job fails when the gate fails. Nothing served changes either way: promotion to serving is a deploy

The loop: the drift alarm says the world moved, new tickets go into the data, the pull request runs the gate on the same split, the deploy serves the winner, and the service starts measuring drift against the new profile.

What exists after today, AWS track

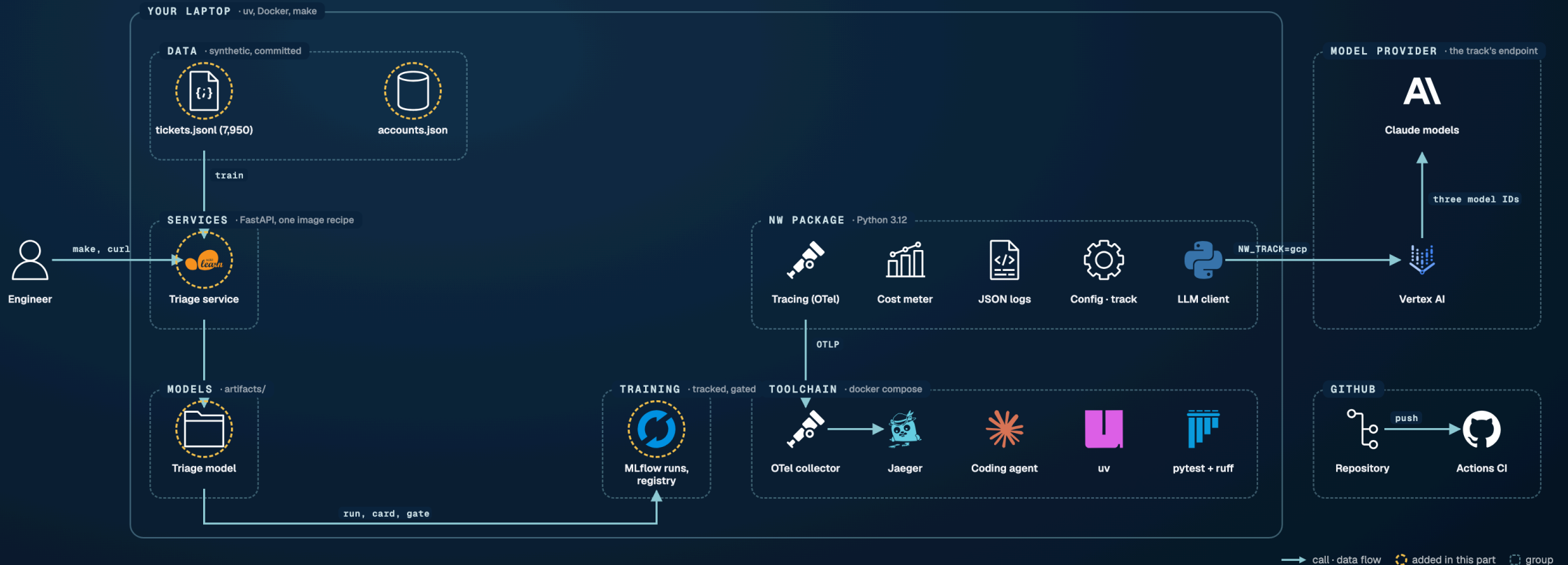
MACHINE LEARNING IN PRODUCTION · AWS



Your laptop: the committed tickets and the production summary, the triage service with its artifact, data profile and card, the run log, the nw package and the local toolchain. Bedrock stays the track's endpoint; nothing today calls it. Ringed: what this project adds.

What exists after today, Google Cloud track

MACHINE LEARNING IN PRODUCTION · GOOGLE CLOUD



Same service, same artifact, same toolchain; Vertex AI is the endpoint. The triage service does not know which track it is on.

Acceptance criteria

1. The data passes its contract and the profile ships in the artifact
2. Same ticket, same features, before and after reload; a corrupt artifact is refused
3. P0 recall at or above 0.8 on the test split, by the threshold rule
4. The candidate passed the gate: `latest`, the alias, the summary and the promotion log agree
5. Service: liveness, readiness, metrics, correlation ID, versioned prediction, drift and shadow

29 passed

what `make session02` prints when you are done

Questions worth asking

- Northwind's Enterprise P0 SLA is 15 minutes. What recall target does that imply, and what precision can the on-call team absorb?
- The threshold was chosen on the validation split. What does `/drift` say the day next quarter's tickets look different, and who gets the alarm?
- The gate compares two models on one test split. What happens when the data changes, and who is allowed to force it?
- What in this service would you have to change if the model became a transformer? What would stay?